

-----大学

本科毕业设计（论文）

基于 .NET 架构的 QoS 感知
Web 服务动态调用平台的设计与实现

学院：_____学院

专业：_____软件工程

学生姓名：_____

学号：_____

指导教师：_____

职称：_____

二〇二六年九月

摘 要

随着分布式软件系统规模不断扩大，同一业务能力往往由多个接口相近但服务质量不同的提供者共同承担。传统应用通常在源代码或配置文件中固定服务地址，当网络状态、响应速度、运行负载或服务可用性发生变化时，调用方缺乏主动选择和快速恢复能力。针对这一问题，本文设计并实现了一套基于 .NET 10 的 QoS 感知 Web 服务动态调用平台。系统把天气查询、物流报价、汇率换算和消息通知抽象为四类业务能力，为每类能力注册多个候选服务，并依据可用性、成功率、响应时间、调用成本和当前负载五项质量指标进行归一化和加权评分，在运行时选择综合得分最优的服务端点。

平台采用领域层、应用层、基础设施层、Web 展示层和模拟服务层相互分离的软件架构。组合调用编排器允许用户一次选择一至四类能力，并使用异步并行机制缩短整体响应时间。系统设置两秒调用超时，候选失败后按照排名依次故障转移；同一端点连续失败两次时熔断二十秒，后台健康监控每十秒探测服务状态，将离线端点从候选池中排除。调用结果会反向更新内存 QoS 快照，最近一百次组合请求则以 JSON 形式持久保存。

为体现传统 Web Service 动态调用主题，系统除 REST/JSON 外还实现了 SOAP 1.1 与 WSDL。SOAP 客户端不依赖编译期生成的代理类，而是在首次调用时下载并解析 WSDL，校验操作定义，根据服务注册表中的地址与 SOAP Action 动态构造 XML Envelope，并将响应转换为统一结果。系统使用 Razor Pages 提供可视化控制台，同时开放组合调用 API。部署方面采用 Docker Compose 将调度平台和服务提供者封装为两个 Linux 容器，并通过命名卷保存调用历史。测试结果表明，系统能够正确完成 QoS 排名、REST/SOAP 动态调用、并行编排、健康过滤、故障转移和熔断恢复，达到预定的软件工程设计目标。

关键词：Web Service；QoS；动态调用；ASP.NET Core；SOAP；WSDL；服务组合；Docker

ABSTRACT

As distributed software systems grow, the same business capability is often supplied by multiple endpoints with different quality characteristics. Statically binding a client to one endpoint makes the application vulnerable to latency changes, temporary failures and load fluctuations. This thesis designs and implements a QoS-aware dynamic Web service invocation platform based on .NET 10. Four capability categories—weather, shipping, currency conversion and notification—are registered with multiple candidate providers. Availability, success rate, response time, invocation cost and load are normalized and combined through configurable weights so that the most suitable endpoint can be selected at runtime.

The system follows a layered architecture and supports parallel orchestration of one to four service categories. Timeout control, ranked failover, a circuit breaker and a background health monitor improve resilience. Both REST/JSON and SOAP 1.1 are supported. Instead of using a generated SOAP proxy, the client retrieves and validates WSDL at runtime, constructs an XML envelope according to registry metadata, and converts the SOAP result into the unified response model. A Razor Pages console explains rankings and invocation attempts, while Docker Compose provides a repeatable two-container deployment.

Key words: Web Service; Quality of Service; Dynamic Invocation; ASP.NET Core; SOAP; WSDL; Service Composition; Docker

目 录

第1章 绪论

1.1 研究背景与意义 1.2 国内外相关技术 1.3 研究内容 1.4 论文结构

第2章 相关理论与关键技术

2.1 Web 服务技术 2.2 QoS 服务评价 2.3 .NET 与 ASP.NET Core 2.4 弹性设计 2.5 容器化

第3章 需求分析

3.1 业务需求 3.2 功能需求 3.3 非功能需求 3.4 用例与可行性

第4章 系统总体设计

4.1 设计原则 4.2 分层架构 4.3 模块划分 4.4 数据模型 4.5 部署设计

第5章 QoS 动态选择算法设计

5.1 指标体系 5.2 归一化 5.3 加权评分 5.4 选择流程 5.5 复杂度分析

第6章 系统详细设计与实现

6.1 服务注册 6.2 组合编排 6.3 REST 调用 6.4 SOAP/WSDL 调用 6.5 健康检查与熔断 6.6 历史与页面

第7章 系统测试

7.1 测试环境 7.2 单元测试 7.3 集成测试 7.4 端到端测试 7.5 结果分析

第8章 部署与运行维护

第9章 总结与展望

参考文献 致谢

第1章 绪论

1.1 研究背景与意义

面向服务架构强调通过标准化契约把业务能力封装为可复用服务。调用方只关心服务所提供的功能和公开接口，而不需要了解内部实现。随着互联网应用、云计算和企业信息化的发展，一个抽象能力通常存在多个可替换的提供者。例如天气信息可以来自不同数据源，物流报价可以由多家承运商给出，消息发送也可以选择不同通道。这些服务虽然完成相似功能，但在响应速度、稳定性、价格和负载方面并不相同。

如果系统始终调用固定地址，一旦该端点变慢或不可用，业务请求就会超时甚至整体失败。简单地在配置中准备备用地址虽然能够处理部分故障，却无法根据当前状态主动选择，也难以解释选择依据。服务质量即 QoS 为解决这一问题提供了量化基础。通过采集服务可用性、成功率、响应时间、成本和负载等指标，调用方可以把“哪个服务更合适”转化为可计算、可配置和可验证的决策。

本课题的工程意义体现在三个方面。第一，将服务发现、质量评价、动态绑定和故障恢复组织成完整闭环，避免业务代码直接依赖具体提供者。第二，利用异步并行编排降低组合服务的总体等待时间，并通过健康检查与熔断控制故障影响范围。第三，在现代 REST 技术之外保留 SOAP/WSDL 动态调用，使传统 Web Service 思想与现代 .NET、容器化部署形成直观对照，具有较好的教学和演示价值。

1.2 国内外相关技术概述

早期 Web Service 体系以 SOAP、WSDL 和 UDDI 为代表。SOAP 定义基于 XML 的消息封装，WSDL 描述服务操作、消息、绑定与地址，UDDI 则面向服务注册和发现。该体系具有契约明确、跨语言和工具支持成熟等特点，在企业集成领域得到广泛应用。随着 Web 应用趋向轻量化，REST 借助 HTTP 资源语义和 JSON 数据格式降低了通信复杂度，逐渐成为公共 API 和微服务的常用选择。

无论采用 SOAP 还是 REST，多候选服务的选择都可以归结为多属性决策问题。研究中常见方法包括简单加权法、层次分析法、模糊综合评价、TOPSIS、遗传算法以

及基于历史反馈或机器学习的策略。复杂算法能够描述更多不确定性，但会提高实现和解释成本。本课题面向本科软件工程实践，采用透明的归一化加权模型，使每个候选的总分都能分解到具体指标，便于测试、展示和后续替换算法。

1.3 研究内容

本文围绕“注册—评价—选择—调用—反馈”主线展开。首先建立四类业务服务和十个候选端点，构造可控的时延、错误率、成本和负载差异。其次设计五维 QoS 模型与动态排名算法，实现用户可调权重。再次设计组合调用编排器，使不同类别并行执行，同类候选按排名顺序故障转移。随后加入后台健康探测、连续失败熔断、调用历史持久化和可视化页面。最后实现 WSDL 运行时解析与 SOAP 调用，并使用 Docker Compose 完成可重复部署。

1.4 论文结构

全文共九章。第一章介绍研究背景、意义和主要内容；第二章阐述 Web 服务、QoS、.NET、弹性设计与容器技术；第三章完成业务、功能和非功能需求分析；第四章给出总体架构、模块和数据设计；第五章重点说明 QoS 归一化及选择算法；第六章描述各模块的实现；第七章介绍测试方案和结果；第八章说明容器化部署与维护；第九章总结成果并提出展望。

第2章 相关理论与关键技术

2.1 Web 服务与协议无关调用

Web 服务通过网络暴露可机器调用的业务能力。REST 服务通常把资源映射到 URI，利用 GET、POST 等 HTTP 方法表达操作，并以 JSON 传输数据。SOAP 服务则使用 XML Envelope 封装请求和响应，通过 SOAP Action 标识操作。WSDL 文档以 XML Schema 描述数据类型，以 message、portType、binding 和 service 描述从抽象操作到实际地址的映射。

动态调用的关键并非“使用反射调用某个方法”，而是调用方在运行时根据注册元数据确定协议、地址与契约。本文在 `ServiceEndpoint` 中保存协议类型、服务地址、WSDL 地址和 SOAP Action。统一调用器收到端点后选择 REST 或 SOAP 路径，对上层始终返回相同形式的结果。因此编排器不需要知道候选采用何种协议，实现了业务选择与通信细节解耦。

2.2 QoS 服务评价

QoS 描述服务非功能属性。可用性表示服务在给定时间范围内能够正常响应的概率；成功率来自最近调用反馈；响应时间反映用户等待成本；调用成本用于模拟商业计费；负载表示资源繁忙程度。由于各指标量纲和方向不同，不能直接相加，必须先进行无量纲化处理。本文选择候选集合内部的最小—最大归一化，既便于实现，也能使当前候选之间的差异充分显现。

2.3 .NET 10 与 ASP.NET Core

.NET 提供跨平台运行时、C# 语言、依赖注入、配置、日志和异步任务模型。ASP.NET Core 内置 Kestrel Web 服务器，可同时承载 Razor Pages 和 HTTP API。`HttpClientFactory` 统一管理 `HttpClient` 生命周期，避免频繁创建连接带来的端口耗尽，并为不同用途配置独立超时。本文使用强类型记录表达领域数据，通过接口隔离应用层与基础设施层，通过依赖注入组合服务。

2.4 异步、超时、熔断与健康检查

异步 I/O 能够在等待网络响应时释放线程。组合请求将多个类别映射为任务并交给 `Task.WhenAll`，从而使总耗时接近最慢的单项，而不是各项耗时之和。超时限制单次等待上界；故障转移让调用器尝试下一候选；熔断器在连续错误时暂时拒绝继续访问故障端点，避免故障放大；健康检查则从外部周期性判断 `Provider` 是否在线。四种机制关注点不同，组合使用才能形成较完整的弹性策略。

2.5 Docker 与可重复部署

容器镜像将应用、运行时和启动命令封装在一致环境中。本文采用多阶段 Dockerfile: SDK 镜像负责恢复与发布, 较小的 ASP.NET Runtime 镜像负责最终运行。Compose 定义 Web 和 Provider 两个服务、端口、内部网络、环境变量和命名卷。Web 容器通过服务名 providers 访问 Provider, 宿主机则通过 5200 和 5201 端口展示系统。该方式降低了演示前的环境配置成本。

第3章 需求分析

3.1 业务需求

系统面向教学演示用户和服务管理员。演示用户希望选择若干业务能力、设置质量偏好并发起请求, 同时清楚看到系统选择了哪个候选、为什么选择以及失败时如何切换。服务管理员希望观察健康与熔断状态, 通过故障注入验证容错行为, 并能用统一方式启动和停止整套环境。

3.2 功能需求

编号	功能	说明
F01	服务注册	维护类别、名称、地址、协议、成本、启用状态和 SOAP 元数据
F02	QoS 评分	按五项指标和用户权重计算排名及分项得分
F03	动态调用	运行时选择 REST 或 SOAP, 不在业务流程中固定提供者
F04	组合编排	一次选择一至四类服务并行调用, 允许部分成功
F05	故障恢复	超时、候选切换、熔断、健康过滤
F06	结果解释	显示候选排名、分数、实际端点、尝试次数和耗时
F07	历史记录	保存最近一百次组合调用并在页面展示
F08	运维部署	提供健康接口、容器镜像和一键启停命令

3.3 非功能需求

可靠性方面，任一候选失败不应直接造成同类请求失败，只要仍有可用候选就应继续尝试；某一类别全部失败也不应取消其他类别。性能方面，组合调用必须并行，单次网络调用最长等待两秒。可维护性方面，各层依赖方向清晰，领域和应用代码不依赖具体 Web 框架。可测试性方面，评分器、编排器、注册表和熔断器应可通过替身对象独立验证。可部署性方面，系统应能在具有 Docker Desktop 的 Windows 环境中用一条命令启动。

3.4 可行性分析

技术上，.NET 10 已提供 Web、异步、XML 和依赖注入所需能力，SOAP 动态调用可直接使用 XDocument 与 HttpClient 实现，无需采购商业组件。经济上，开发工具和基础镜像均可免费用于学习，模拟 Provider 不产生第三方调用费用。操作上，Web 页面将复杂选择过程可视化，Docker Compose 屏蔽多进程启动细节。因此项目在技术、经济和操作三个方面均具有可行性。

3.5 角色与用例分析

系统包含演示用户、服务管理员和部署维护人员三类角色。演示用户是主要参与者，其目标是提交服务组合请求并理解系统决策。用户进入首页后选择天气、物流、汇率或通知，可单选也可多选；随后填写业务参数并调整质量权重。提交后系统返回关联标识、并行总耗时、每类服务候选排名、被选端点、实际尝试顺序和业务结果。若输入权重和不是百分之百，页面必须阻止调用并给出明确提示。

服务管理员负责观察和验证运行状态。他可以查看每个端点最近的健康结果和熔断状态，也可调用故障注入接口使指定候选持续返回错误，再观察最高分候选失败后是否切换到下一名。部署维护人员负责启动 Docker Desktop、构建镜像、查看容器状态与日志、停止服务以及决定是否删除历史卷。三类角色权限在演示系统中没有使用账号隔离，但在需求描述中明确区分，有利于未来加入授权模型。

3.6 典型业务流程

以四类组合调用为例，用户输入城市、物流起终点、换算金额和通知收件人，然后保持默认权重并提交。Web 层将表单转换为四个 ServiceTask，并生成全局唯一关联 ID。编排器并行启动四条类别流程，每条流程独立过滤候选、计算排名、选择协议并调用。天气可能选中 REST 或 SOAP，物流、汇率和通知使用 REST。某条流程发生错误时只在自身候选集合内切换，其余流程继续执行。全部任务完成后，编排器汇总成功与失败，计算总耗时，写入历史并返回页面。

故障流程与正常流程同等重要。若 Provider 整体关闭，后台监控在下一轮检查中把全部端点标为离线，新请求不会继续等待这些地址；若只有一个具体端点持续返回五百错误，健康接口仍可能正常，但熔断器会在两次失败后隔离该端点；若网络错误只出现一次，则编排器完成本次故障转移，但不立即熔断，避免偶发抖动造成过度保护。这些流程体现了系统对不同故障粒度的区分。

3.7 数据与接口需求

组合 API 接收 categories、payload 和 weights。categories 为一至四个不重复的类别标识；payload 是字符串键值集合，不同 Provider 从中读取所需字段；weights 可省略，省略时使用默认配置。返回数据包含 correlationId、results 和 elapsed。每个结果包含 categoryId、endpointId、endpointName、success、data、ranking、attempts 和 error。该结构既适合页面展示，也适合其他程序进行二次处理。

系统不保存真实个人敏感信息。通知收件人和业务载荷只参与本次模拟调用，不进入历史摘要。历史文件仅保存关联 ID、时间、类别、成功数量和耗时。日志也不主动输出完整载荷。这一约束符合最小化存储原则，能够降低演示数据泄露风险。若未来接入真实服务，则还应对凭据、令牌、手机号和邮件地址实施加密与脱敏。

3.8 验收标准

功能验收要求四类服务均可选择，每类候选数量与设计一致，默认权重和自定义权重均能产生确定排名，REST 与 SOAP 都能返回统一结果，故障候选能够切换，历

史能跨容器重建保留。性能验收要求四类组合调用采用并行方式，总耗时不等于四项简单相加。可靠性验收要求 Provider 离线不会导致应用崩溃，单类失败不会取消其他结果。工程验收要求解决方案可编译、自动化测试通过、文档与代码一致，并能在新环境中依据部署说明启动。

第4章 系统总体设计

4.1 设计原则

系统遵循单一职责、依赖倒置、显式契约和可观察失败原则。领域层仅保存稳定概念；应用层描述用例；基础设施层处理 HTTP、XML、文件和后台任务；Web 层负责输入输出。服务选择结果不仅返回最终数据，还保留排名和尝试链路，使动态行为能够被解释和审计。

4.2 分层架构



Domain 项目定义端点、权重、快照、评分和结果；Application 项目定义接口并实现评分与组合编排；Infrastructure 项目实现注册表、动态调用、指标、熔断、健康和历史；Web 项目提供交互页面与聚合 API；MockProviders 项目提供十个具有不同质量特征的模拟候选。项目引用只沿规定方向建立，避免循环依赖。

4.3 模块划分

服务注册模块按类别查询候选，并通过 PROVIDER_BASE_URL 适配本机或容器网络。QoS 模块把快照和权重转换为有序 ServiceScore。编排模块验证请求、创建并行任务、执行故障转移并聚合结果。调用模块封装 REST 和 SOAP 细节。弹性模块维护健康与熔断状态。历史模块把摘要写入 JSON。展示模块将这些信息组织成表单、排名、调用结果和运行状态。

4.4 核心数据模型

ServiceEndpoint 包含 Id、CategoryId、Name、Address、CostPerCall、Enabled、Protocol、WsdAddress 和 SoapAction。QosSnapshot 保存 Availability、SuccessRate、AverageResponseTimeMs、Load 和采集时间。QosWeights 保存五项权重并负责校验。ServiceScore 同时保存总分、分项贡献与排名。CompositeRequest 包含关联 ID、任务集合和权重；CompositeResult 包含各类别结果与总耗时；Attempt 记录每次候选尝试及错误类型。

4.5 部署设计

Compose 网络中包含 web 与 providers 两个容器。providers 在容器内监听 8080，对宿主映射 5201；web 同样监听 8080，对宿主映射 5200。Web 的 PROVIDER_BASE_URL 设置为 http://providers:8080，因而不依赖宿主端口回环。调用历史目录挂载到 qos-history 命名卷，容器重建后数据仍可保留。

4.6 架构方案比较与决策

项目设计阶段比较了单体应用、完全微服务和分层双宿主三种方案。单体应用把页面、调度与所有 Provider 放在同一进程，启动最简单，但无法真实体现网络动态调用，健康检查和容器网络也失去意义。完全微服务方案为十个候选各建一个进程或容器，隔离最彻底，却会产生大量重复项目、端口和构建配置，偏离本科课题对核心算法的关注。最终采用分层双宿主方案：Web 与调度在一个宿主，全部模拟

Provider 在另一个宿主，但每个候选仍拥有独立 URI 和质量画像。它在真实性与复杂度之间取得平衡，并保留将候选拆分为独立服务的能力。

在存储方案上比较了内存、JSON 和关系数据库。QoS 运行状态变化频繁但无需长期保存，使用线程安全内存结构最合适；调用历史需要跨重启保留，但数据结构简单且规模限制为一百条，JSON 足够直观；服务注册表为了便于阅读和稳定测试也保留在代码和配置中。若引入数据库，虽然能支持查询与并发，却增加迁移、连接字符串和容器依赖，因此作为后续增强而非当前必需。

在 SOAP 客户端方案上比较了静态服务引用、运行时代码生成和直接 XML 调用。静态服务引用开发便利，但代理在编译期固定，难以突出动态发现；运行时代码生成最接近通用代理，却需要复杂的元数据导入、编译和类型映射；直接 XML 调用通过解析 WSDL 验证契约，再按注册元数据构造消息，能够清晰展示全过程，且足以支持本项目的单一操作。因此本文选择第三种方案。

4.7 接口隔离与依赖关系

IServiceRegistry、IQosMetricsStore、IQosScorer、IServiceInvoker、ICircuitBreaker、IEndpointHealthMonitor 和 ICallHistoryStore 构成应用层边界。编排器只依赖这些抽象，单元测试可分别提供内存替身。基础设施层实现接口并由 ServiceCollectionExtensions 注册生命周期。注册表、指标、熔断器、历史和健康监控采用单例，使同一进程内状态一致；调用器采用瞬态，但内部 HttpClient 由工厂管理。后台健康监控既作为单例接口被查询，也作为 HostedService 启动，依赖注入确保二者指向同一实例。

4.8 并发与一致性设计

组合调用天然产生并发，因此共享状态必须线程安全。指标存储和熔断器使用 ConcurrentDictionary，更新通过 AddOrUpdate 完成，避免普通字典在并发写入时损坏。历史存储使用进程内锁把加载、插入、裁剪和写文件视为一个临界区，保证 JSON 内容不会被两个并行请求交叉覆盖。WSDL 校验缓存也使用并发字典，使多个请求同时首次访问时不会破坏缓存状态。

当前一致性边界限定在单个 Web 实例。若部署多个 Web 副本，每个副本会拥有独立指标和熔断状态，文件卷并发写入也存在冲突。因此多副本场景应将指标与熔断状态迁移到 Redis，将历史迁移到数据库，并使用分布式锁或事务。明确这一边界是软件工程设计的重要内容，它说明当前方案对目标场景是正确的，但不是无限扩展的。

第5章 QoS 动态选择算法设计

5.1 指标体系

Availability 来自基准可用率和健康探测，SuccessRate 根据实际成功次数与调用次数计算，ResponseTime 使用累计平均耗时，CostPerCall 在注册时设定，Load 为模拟的当前负载。前两项属于效益型指标，后三项属于成本型指标。默认权重依次为 0.35、0.25、0.20、0.10 和 0.10，反映系统优先保证可用与成功，其次考虑速度，最后权衡成本和负载。

5.2 指标归一化

设候选集合中某项指标值为 x ，最小值和最大值为 $\min$ 与 $\max$ 。效益型指标归一化为 $(x-\min)/(\max-\min)$ ，成本型指标归一化为 $1-(x-\min)/(\max-\min)$ 。当 $\max$ 与 $\min$ 相等时，所有候选在该维度没有区分度，统一计为 1，避免除零。该策略使用当前候选集合的相对差异，计算简单且结果落在零到一之间。

5.3 加权评分

```
Score = wA·Availability + wS·SuccessRate
      + wR·ResponseSpeed + wC·CostEfficiency
      + wL·LoadCapacity
约束：wA + wS + wR + wC + wL = 1，且每项权重位于 [0,1]。
```

系统同时保存每项加权后的贡献值，而不是只保存总分。页面能够显示最终排名，后续也可扩展成雷达图或趋势图。当总分相同时按端点名称升序排列，保证相同输入下结果稳定，减少测试中的随机性。

5.4 动态选择流程

编排器先查询指定类别的所有端点，依次调用熔断器和健康监控进行过滤，再为剩余候选读取快照并评分。排名第一者先被调用。若调用成功，编排器记录成功、耗

时并重置其连续失败数；若发生 `HttpRequestException`、`TaskCanceledException` 或 `TimeoutException`，则记录失败并增加熔断计数，然后尝试下一名。所有候选耗尽后，该类别生成结构化失败结果。

5.5 算法复杂度与适用性

设某类别候选数为 n 。指标极值计算和得分计算均为 $O(n)$ ，排序为 $O(n \log n)$ ，所以总体复杂度为 $O(n \log n)$ ，空间复杂度为 $O(n)$ 。本项目每类只有二至三个候选，算法开销可以忽略。即使扩展到数百个端点，主要成本仍是网络调用而不是评分。简单加权法的限制是权重由用户显式给出且指标之间被视为相互独立，但其透明性和可解释性非常适合教学系统。

第6章 系统详细设计与实现

6.1 服务注册表

`InMemoryServiceRegistry` 为每个候选分配稳定 `Guid`，保存类别、名称、地址、成本和协议。REST 端点路径位于 `/api/providers/{category}/{provider}`。SOAP 天气端点路径为 `/soap/legacy-weather`，WSDL 使用同一路径的 `wsdl` 查询参数。注册表从配置或环境变量读取 `Provider` 基地址，因此本机运行时使用 `localhost:5201`，容器运行时自动切换为 `providers:8080`。

6.2 组合调用编排

`CompositeOrchestrator` 是应用核心。`ExecuteAsync` 首先校验任务数必须为一至四，且不能重复类别，同时校验权重和。随后使用 LINQ 把每个 `ServiceTask` 投影为 `ExecuteOneAsync` 任务，并由 `Task.WhenAll` 并行等待。每个任务独立获取候选、评分和调用，所以单类失败不会抛出并取消整个请求。全部完成后，编排器生成 `CompositeResult`，记录总耗时并写入历史存储。

6.3 REST 动态调用

REST 路径通过 HttpClientFactory 获取名为 dynamic-services 的客户端，将任务 Payload 序列化为 JSON 并 POST 到运行时端点。响应必须具有成功状态码，随后被反序列化为 JsonElement，再格式化为统一字符串。调用器本身不知道天气、物流等具体业务，只处理地址和载荷，这使新 REST 候选可以仅通过注册信息接入。

6.4 SOAP/WSDL 动态调用

SOAP 实现是本课题区别于普通 REST 聚合器的关键。首次调用某 WSDL 地址时，调用器使用 HTTP GET 下载文档，以 XDocument 解析 WSDL 命名空间，并搜索名称为 QueryWeather 的 operation。若契约中不存在该操作，则拒绝调用。验证结果缓存在并发字典中，后续调用不必反复下载。

契约通过后，调用器创建 SOAP 1.1 Envelope，在 Body 中加入 weather 命名空间的 QueryWeather 元素和 city 参数，以 text/xml 发送，并设置注册表提供的 SOAPAction。响应解析时先检查 Fault；存在 Fault 就转化为 HttpRequestException，从而进入统一故障转移流程；否则提取 QueryWeatherResult，并包装成包含 protocol、wsdl 和 result 的 JSON。该方案没有编译期服务引用，真正体现了运行时契约发现与绑定。

6.5 健康检查、指标反馈与熔断

EndpointHealthMonitor 继承 BackgroundService，每十秒按服务基地址分组访问 /health，并把结果写入线程安全字典。初次检查前采用乐观可用，检查失败后相关端点均标记离线。InMemoryQosMetricsStore 使用 ConcurrentDictionary 保存调用次数、成功次数和平均耗时，通过 AddOrUpdate 原子更新。

InMemoryCircuitBreaker 维护连续失败数与 OpenUntil。第一次失败只记录；第二次连续失败把电路打开二十秒。打开期间 CanExecute 返回 false。期限到达后允许一次正常请求，相当于简化的半开状态：若成功则计数归零，若失败则再次打开。健康检查解决“宿主是否在线”，熔断器解决“具体端点是否持续失败”，两者相互补充。

6.6 历史存储与展示页面

JsonCallHistoryStore 用进程锁保护读写，把关联 ID、完成时间、类别数、成功数、总耗时和类别列表保存到 App_Data/call-history.json，仅保留最近一百条。容器将该目录映射到命名卷。Razor 页面提供服务复选框、输入字段和五项权重，执行后展示并行总耗时、各类别排名、实际选择、每次尝试、服务响应、历史、熔断和健康状态。页面由服务端渲染，不需要 Node.js 前端工具链。

6.7 模拟 Provider

Provider 通过 fast、balanced、economy 和 reliable 四种质量画像控制延迟和失败率。例如 fast 响应快但负载与成本较高，economy 价格低但延迟与失败率较高。管理接口可以强制指定 REST 端点失败，用于稳定复现故障转移。SOAP Provider 生成标准 WSDL，并对 SOAP 请求解析 city 参数，返回带城市、温度和来源说明的 XML 响应。

6.8 异常处理与输入安全

系统在入口和边界分别处理异常。页面模型验证类别数量、类别重复和权重总和；组合 API 对不合法请求返回四百状态，而不是让异常传播为五百。网络层仅捕获预期的请求、取消和超时异常，使编程错误不会被误当成服务故障而静默吞掉。Provider 对未知类别和画像返回四百零四，对模拟故障返回五百零三。SOAP 响应中的 Fault 被识别并转换为网络异常，从而与 REST 非成功状态进入同一故障转移流程。

XML 处理使用内存中的 XDocument，并且 SOAP 请求体由程序构造，不进行字符串拼接，城市内容会被自动转义。历史仅保存摘要，避免把完整通知内容写入磁盘。演示环境只监听本机映射端口，没有配置公开域名。生产化时仍需限制请求体大小、增加身份认证、跨站请求保护、速率限制和 HTTPS，并对 WSDL 来源建立白名单，防止服务器端请求伪造。

6.9 可维护性设计

领域模型采用不可变 record，减少共享数据被意外修改。常量 ServiceCategories 统一类别标识，避免散落的魔法字符串。协议使用 ServiceProtocol 枚举，使新增协议时编译器能够提示遗漏分支。端点标识使用固定 Guid，使测试、历史和状态关联保持稳定。主要阈值集中在实现中，例如两秒超时、十秒健康周期、两次失败和二十秒熔断，后续可进一步移动到配置文件。

代码结构与文档采用相同术语，CompositeOrchestrator、QosScorer、ServiceEndpoint 等名称直接反映职责。README 面向使用者，PLAN 记录设计状态，UML 描述结构和流程，DEPLOYMENT 专注环境操作。通过让文档与代码共同演进，降低了毕业设计常见的“论文设计与最终程序不一致”问题。

第7章 系统测试

7.1 测试环境与策略

测试环境为 Windows x64、.NET SDK 10.0.401、ASP.NET Core Runtime 10.0.12、WSL 2.7.14、Docker Engine 29.8.0 和 Docker Compose 5.5.1。测试分为单元测试、集成测试和端到端测试。单元测试隔离评分与编排逻辑；集成测试验证真实注册数据；端到端测试启动实际进程或容器，通过 HTTP 验证从入口到 Provider 的完整链路。

7.2 单元测试

QosScorerTests 验证速度权重为一时最快端点排名第一，成本权重为一时最低成本端点排名第一，以及权重和不为一时抛出异常。CompositeOrchestratorTests 使用伪注册表、指标、调用器、熔断器、健康监控和历史存储，令第一候选计划性失败，验证编排器调用第二候选并记录两次 Attempt。CircuitBreakerTests 验证连续两次失败打开电路，成功调用能够清零失败计数。

7.3 集成与端到端测试用例

用例	操作	期望结果	结果
T01	查询四类注册候选	数量分别为3、3、2、2	通过
T02	同时调用四类服务	四项成功且并行聚合	通过
T03	强制第一候选失败	自动选择下一候选	通过
T04	连续失败两次	端点进入20秒熔断	通过
T05	关闭 Provider	健康检查标记离线并过滤	通过
T06	GET SOAP WSDL	HTTP 200 且包含 QueryWeather	通过
T07	成本/负载各50%	选择“经典 SOAP 天气”	通过
T08	Compose 构建启动	两个容器 Up，端口可访问	通过

7.4 测试结果分析

解决方案最终编译结果为零错误、零警告，现有十个自动化测试全部通过。四类并行调用实测总耗时约三百毫秒，接近最慢单项而明显小于串行总和。容器环境中 Web 和 Provider 健康接口均返回 healthy，WSDL 返回状态 200。通过把成本与负载权重各设为 0.5，系统在三个天气候选中选中“经典 SOAP 天气”，返回 Success=true 且结果标记 SOAP 1.1，证明 WSDL 解析、SOAP 构造、容器网络和统一编排链路均有效。

随机故障可能使单次演示结果变化，这是模拟真实 QoS 波动的设计行为。需要确定性演示时，可使用故障注入接口或调整权重。当前测试尚未覆盖高并发压力、长时间稳定性和容器故障自动恢复，这些内容可在后续使用专业压测与可观测工具补充。

7.5 边界与异常测试

权重测试覆盖总和大于一、存在负数以及全部候选指标相同的情况。前两类输入必须被拒绝；指标相同时归一化返回一，避免除零。组合请求测试覆盖零个任务、超

过四个任务和重复类别，均应在调用 Provider 前失败。注册表对未知类别返回空集合，编排器据此生成“所有候选服务均不可用”的结构化结果，而不是空引用异常。

网络边界测试包括 Provider 未启动、端点返回五百零三、请求超过两秒、SOAP 响应缺少结果元素以及 WSDL 不包含目标操作。对于网络和协议故障，调用器抛出可识别异常并触发下一候选；对于契约配置错误，系统保留错误类型以便定位。停止 Provider 后，健康监控能够把端点标记为离线；重新启动并等待下一轮探测后，端点重新参与排名。

7.6 软件质量评价

依据软件质量模型，可以从功能适合性、性能效率、兼容性、易用性、可靠性、安全性、可维护性和可移植性评价本系统。功能适合性通过四类服务、动态评分、双协议和组合 API 得到体现；性能效率通过并行编排实现；兼容性表现在 REST 与 SOAP 共存；易用性由可视化页面和一键启动改善；可靠性由超时、故障转移、健康检查和熔断保障；可维护性来自分层与接口；可移植性则由 .NET 和 Linux 容器提供。

安全性是当前相对薄弱的维度。系统没有账号体系，故障注入接口也没有授权，因此仅适合本机教学环境。数据不包含真实敏感内容、历史采用最小字段、XML 自动转义，提供了基本防护，但不能替代生产安全控制。对质量属性进行客观评价，有助于避免仅以“功能运行成功”作为项目完成标准。

第8章 部署与运行维护

8.1 容器构建

Web 和 Provider 各有独立 Dockerfile。构建阶段复制仓库并恢复对应项目，使用 Release 配置发布；运行阶段仅复制发布结果。docker-compose.yml 定义镜像构建上下文、端口、环境变量、依赖关系、重启策略和命名卷。由于构建上下文包含多个项目，.dockerignore 排除 bin、obj、缓存、Git 和历史数据，减少传输量。

8.2 启停与检查

Windows PowerShell 可能禁止执行未签名 ps1，因此项目提供 start-demo.cmd 和 stop-demo.cmd。启动脚本将 Docker CLI 路径加入环境，检查引擎；若未运行，则启动 Docker Desktop 并最多等待一百二十秒，随后执行 `docker compose up --build -d`。停止脚本执行 `docker compose down`，但保留命名卷。管理员可通过 `docker compose ps` 检查状态，通过 `docker compose logs -f` 查看日志。

8.3 运维注意事项

5200 和 5201 端口必须未被其他程序占用。普通停止不会删除历史数据；只有执行 `docker compose down --volumes` 才会删除命名卷，应谨慎使用。生产环境还应配置 HTTPS、集中日志、密钥管理、认证授权、资源限额和正式数据库。当前系统定位为毕业设计演示，因此优先保证结构清晰和可复现，而不是直接承担生产流量。

第9章 总结与展望

9.1 工作总结

本文完成了一套基于 .NET 的 QoS 感知 Web 服务动态调用平台。系统将四类业务能力和十个候选端点纳入统一注册，利用多维指标完成动态排名，通过协议无关调用器同时支持 REST 和 SOAP，并以并行编排、超时、故障转移、健康检查和熔断提升可靠性。Razor 页面把算法依据与调用链路直观展示，JSON 历史提供基本追溯，Docker Compose 则把多进程环境封装为可重复启动的演示系统。

实践表明，动态服务调用的难点并不只是发送 HTTP 请求，而在于如何把契约、质量、状态、决策、失败和部署视为一个整体。分层架构与接口抽象使评分算法、协议实现和存储方式可以独立演进；可解释评分和尝试记录使系统行为能够被验证；WSDL 运行时发现则使传统 Web Service 主题得到真实体现。项目已经达到软件工程专业本科毕业设计所要求的需求分析、架构设计、编码实现、测试验证和部署交付完整性。

9.2 不足与展望

当前 QoS 基准数据部分为模拟值，成功率和平均响应时间也采用简单累计方式，长时间运行时对近期变化不够敏感。后续可采用滑动窗口或指数加权移动平均。历史使用 JSON 文件，适合单实例演示但不适合多实例并发写入，可替换为 SQLite、PostgreSQL 或时序数据库。评分模型可进一步加入层次分析、TOPSIS 或基于强化学习的自适应权重。

可观测性方面，可集成 OpenTelemetry，把关联 ID 贯穿页面、编排器与 Provider，并输出指标、日志和分布式追踪。协议方面，可扩展更多 SOAP 操作、复杂 XML Schema、WS-Security 或 gRPC。部署方面，可将每个候选拆分为独立容器，在 Kubernetes 中使用服务发现与弹性伸缩。安全方面，应加入身份认证、授权、输入验证、传输加密和凭据托管。通过上述扩展，本系统可从教学原型逐步演进为更接近企业级服务治理平台的实现。

参考文献

- [1] Thomas Erl. Service-Oriented Architecture: Concepts, Technology, and Design[M]. Prentice Hall, 2005.
- [2] W3C. Web Services Description Language (WSDL) 1.1[EB/OL].
<https://www.w3.org/TR/wsdl/>.
- [3] W3C. SOAP Version 1.2 Part 1: Messaging Framework[EB/OL].
<https://www.w3.org/TR/soap12-part1/>.
- [4] Fielding R T. Architectural Styles and the Design of Network-based Software Architectures[D]. University of California, Irvine, 2000.
- [5] Microsoft. ASP.NET Core documentation[EB/OL].
<https://learn.microsoft.com/aspnet/core/>.
- [6] Microsoft. IHttpConnectionFactory with .NET[EB/OL].
<https://learn.microsoft.com/dotnet/core/extensions/httpclient-factory>.
- [7] Microsoft. Background tasks with hosted services in ASP.NET Core[EB/OL].
<https://learn.microsoft.com/aspnet/core/fundamentals/host/hosted-services>.
- [8] Docker Inc. Docker Compose documentation[EB/OL].
<https://docs.docker.com/compose/>.
- [9] ISO/IEC. ISO/IEC 25010 Systems and software quality models[S]. 2011.
- [10] Bass L, Clements P, Kazman R. Software Architecture in Practice[M]. Addison-Wesley, 2021.
- [11] Newman S. Building Microservices: Designing Fine-Grained Systems[M]. O'Reilly Media, 2021.
- [12] Richardson C. Microservices Patterns[M]. Manning Publications, 2018.
- [13] Gamma E, Helm R, Johnson R, Vlissides J. Design Patterns[M]. Addison-Wesley, 1994.

[14] Sommerville I. Software Engineering[M]. Pearson, 2015.

[15] 张海藩, 牟永敏. 软件工程导论[M]. 北京: 清华大学出版社, 2013.

致 谢

本课题从早期 Web Service 动态调用设想出发, 经过需求整理、架构重构、功能实现、自动化测试和容器化部署, 最终形成能够稳定展示的完整系统。在此感谢指导教师在选题、设计和论文写作方面给予的帮助, 感谢软件工程专业课程所提供的需求分析、面向对象设计、软件测试与项目管理知识, 也感谢开源社区提供 .NET、Docker 及相关技术资料。项目实施过程中对旧设计的重新审视, 使作者进一步认识到软件质量不仅体现在功能能否运行, 更体现在结构是否清晰、失败是否可控、行为是否可解释以及环境是否可复现。谨向所有给予支持和建议的老师、同学与家人表示诚挚感谢。